NODE.JS AUTHENTICATION WEB APPLICATION

Project Title: **Node.js Web Application with Authentication**
Course: **Web Programming (Seminar Homework)**

Student Name: Ayman and Abdurrahman
Matric Number: Cw6nlt and GFQ7SD

Live URL: https://nodeview.onrender.com/

It login details:

login = admin@gmail.com

Password = admin123

Linux Server: http://143.47.98.96/app207/

Linux password: Stud207@2025

PROJECT DESCRIPTION

This seminar project focuses on creating a Node.js web application with authentication features such as Register, Login, and Dashboard. The application was built using Express.js, EJS templates, MySQL database (hosted on Railway), bcrypt for password hashing, and deployment on Render cloud hosting.

The aim of this project is to demonstrate practical knowledge of:

- ✔ **Server-side programming with Node.js**
- ✔ **Secure authentication using bcrypt**
- ✔ **Database integration using MySQL**
- ✔ **Cloud deployment using Render**
- ✔ **Responsive UI using Tailwind CSS**

All steps of development are documented with explanations and screenshots across the following sections

This project is a practical implementation of a Node.js web application with user authentication features (Register, Login, Logout, and Dashboard). The main purpose of this work is to understand how server-side web development works using Express.js, EJS templating, MySQL cloud database (Railway), and deployment via Render.com.

In this documentation, I explain all the steps from installation of Node.js, creating a project folder, installing necessary packages, connecting to a cloud database, building authentication (with bcrypt for password hashing), and deploying the final application online. All screenshots are included to show the real implementation on a live server.

The aim of this task is to demonstrate:

- How to create a **Node.js** server using **Express.js**

- How to design dynamic frontend pages using **EJS**

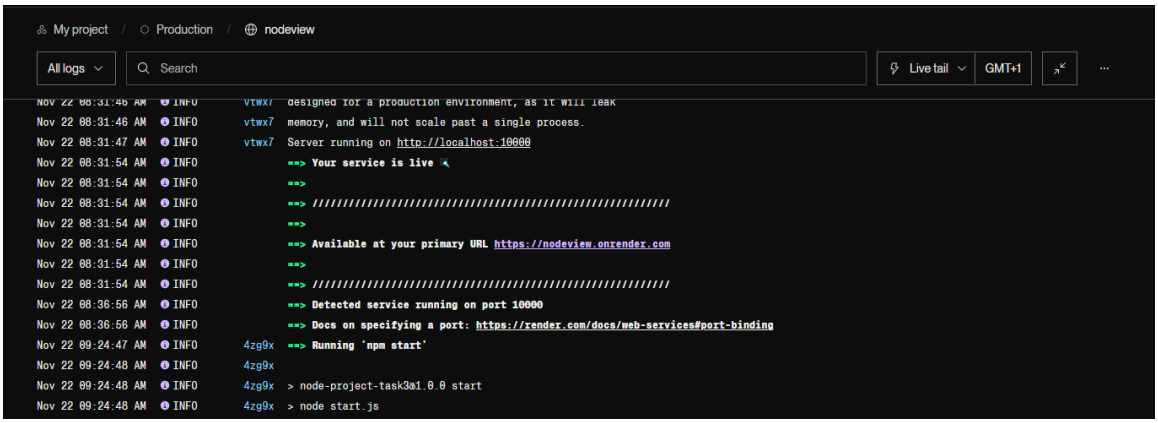
- How to connect Node.js with **MySQL** database on **Railway**

- How to implement secure authentication using **bcrypt**

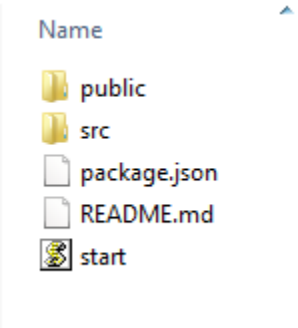
- How to deploy and host a live web application using Render.com

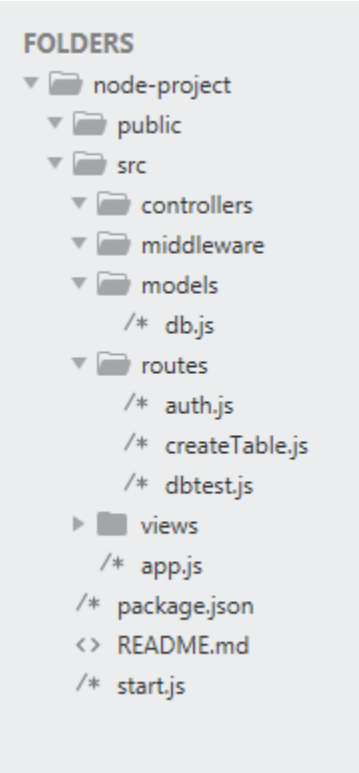
This project is useful for learning full-stack web development, cloud deployment, and backend programming with JavaScript. It also shows how to solve real-world problems such as database connection errors, deployment challenges, authentication logic, and responsive UI design.

Deployment to render screenshot



File structure

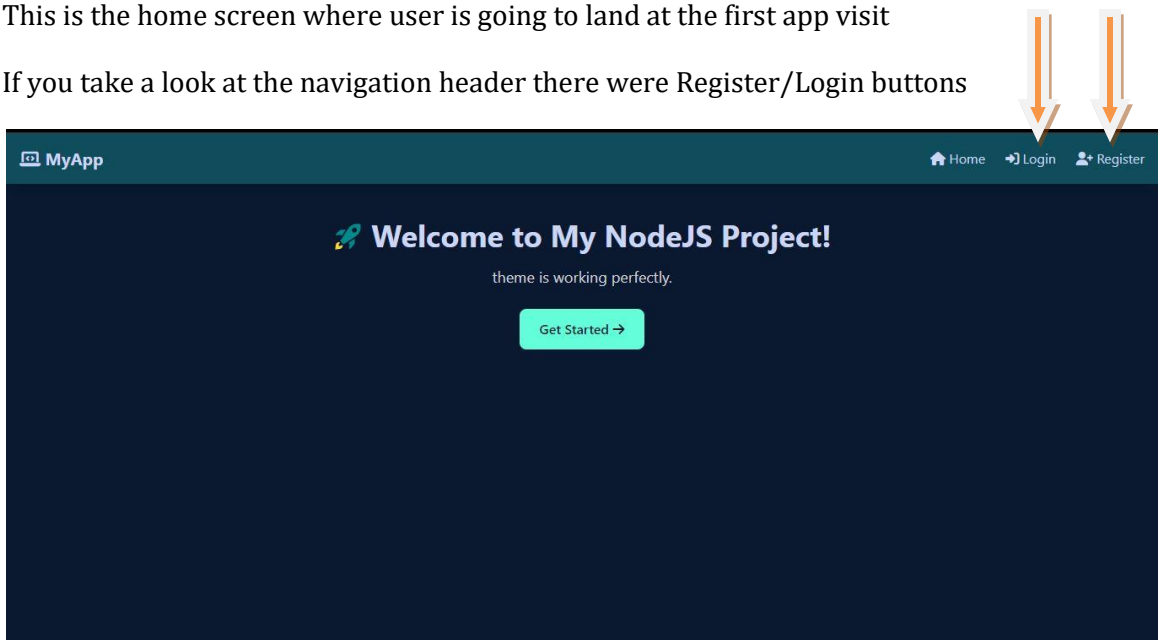




Home screen

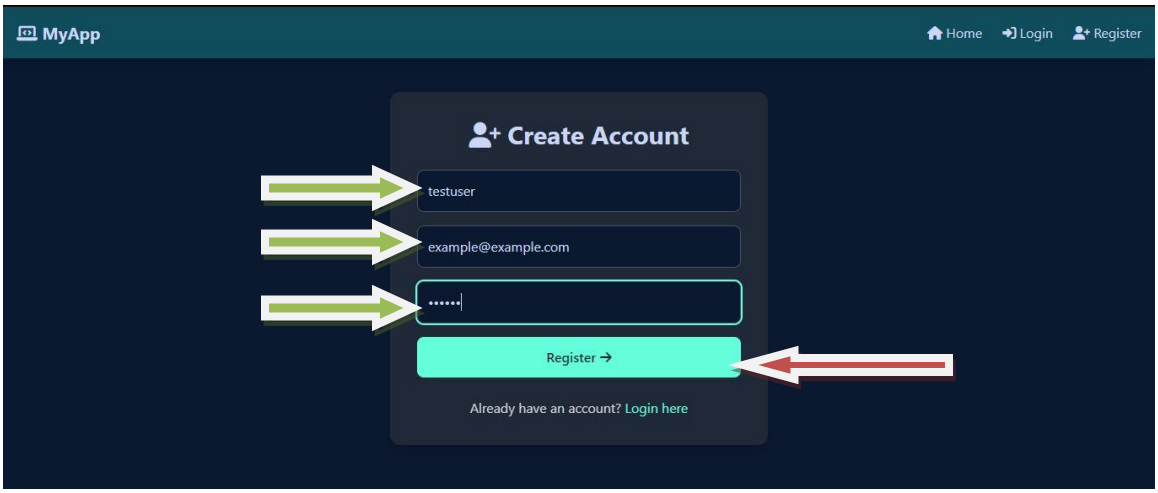
This is the home screen where user is going to land at the first app visit

If you take a look at the navigation header there were Register/Login buttons



Register Page

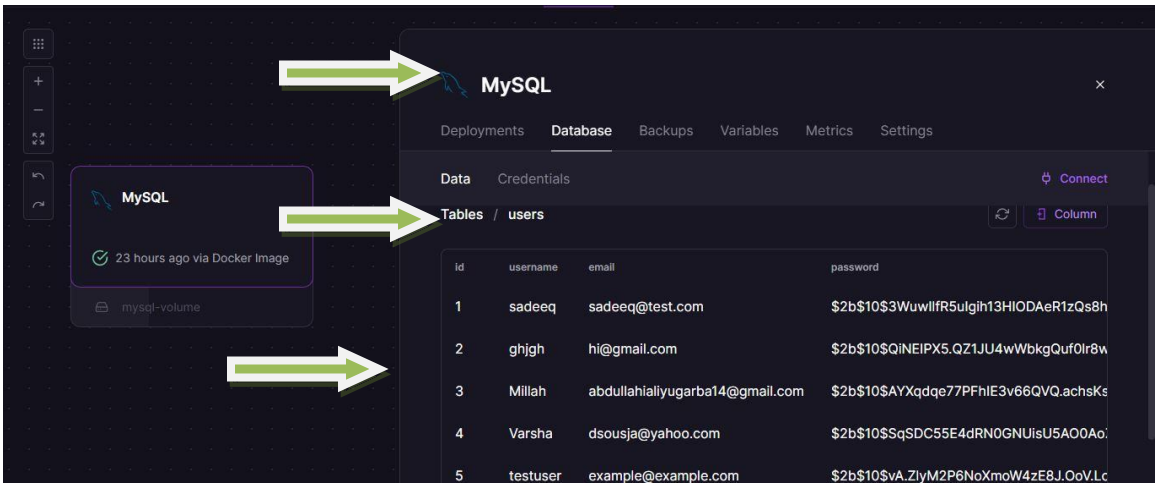
User can register via app/register route using responsive form in register.ejs then after registration he will redirected to login page after saving login information in the database , and also the user password was securely encrypted using bcrypt



Database table (for storing user details)

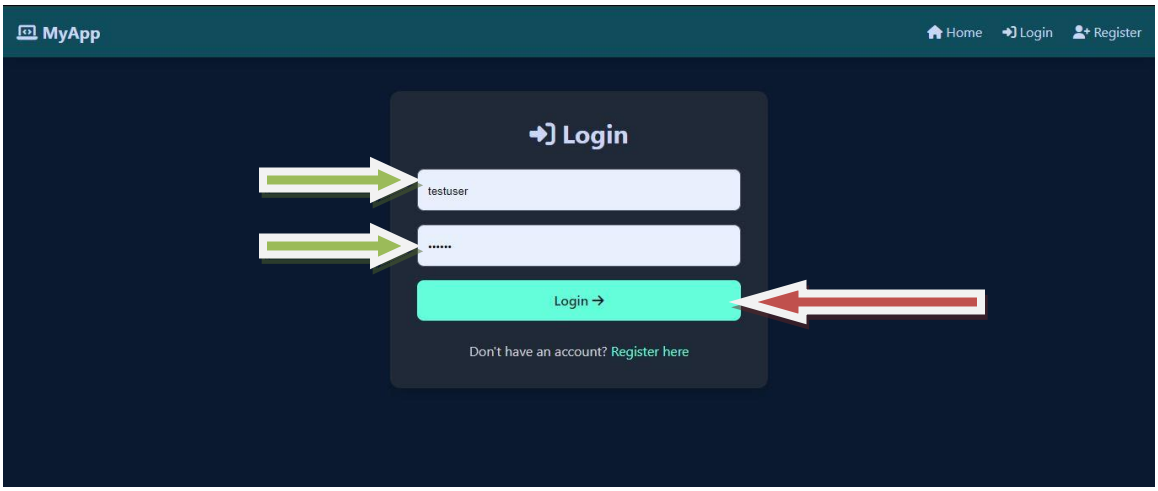
I used Railway MySQL as my database for storing datas from my app , as you can see here are some test datas so far,

I.e when ever user input his details from registeration page , the backend was processing it and encrypting the password to hash value and save the data to the database as you can see here.



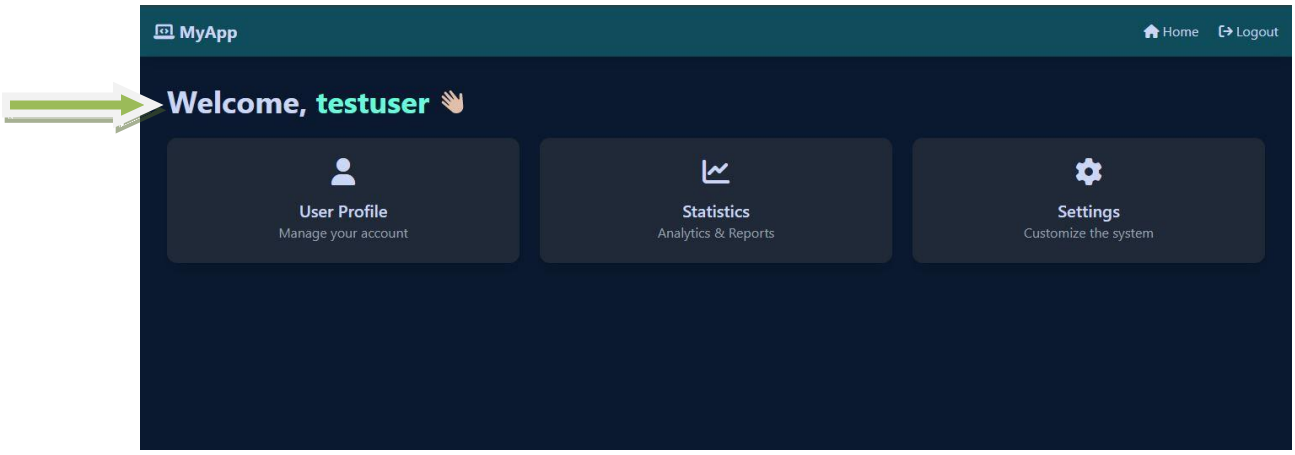
Login page

After user created an account and backend handled the form submission to database , the user was redirected to the login page as you can see here



Dashboard screen

After successful Login the user is going to redirect to his dashboard after checking the database and the user is valid this logic I handle by authentication which carried out from **auth,js** File



3. Project Structure

To begin the development of this Node.js authentication web application, I created a dedicated project folder on my computer (or on GitHub Codespace). Inside this folder, I initialized the Node.js project using the command:

npm init –y

This generated the `package.json` file which stores project information and dependencies. Next, I installed the required dependencies such as Express.js, EJS, MySQL2, bcrypt, and dotenv. I also organized the project into a clear folder structure including:

- `src/app.js` (main server file)

- `src/routes/` (all route files)

- `src/views/` (EJS pages)

- `src/models/` (database connection)

- `public/` (static CSS, JS, and assets)

After creating the basic structure, I set up Express.js and configured EJS as the template engine. Additionally, I created an `.env` file to store database credentials securely for Railway (MySQL) and Render deployment. This made the project well organized, easy to understand, and ready for further development.

Project Folder Structure Example:

```
project-folder/
 │── src/
 │    │── app.js
 │    │── routes/
 │    │── views/
 │    │── models/
 │── public/
 │── package.json
 │── .env
 │── README.md
```

4. Installation of Node.js

To run a Node.js application, Node.js must be installed on the system. In my case, I installed Node.js (version 18+) from the official website: https://nodejs.org

After installation, I opened the terminal (or Git Bash / CMD) and confirmed the installation using the following commands:

node -v

npm -v

These commands are used to verify the version of Node.js and npm (Node Package Manager). Once both versions were displayed, it confirmed that Node.js was successfully installed.

If Node.js is not installed, it can be downloaded manually from the official website or installed using commands such as:

Windows Installer:

Download the .msi installer and follow the setup steps.

For Linux (Ubuntu):

sudo apt update

sudo apt install nodejs

sudo apt install npm

For Mac:

brew install node

After installation, Node.js allows us to run JavaScript on the server, install packages, and manage our authentication web application. This step is very important because without Node.js, the project will not work.

5. Installing Dependencies

After creating the project folder and initializing Node.js using `npm init -y`, the next step was to install all necessary dependencies required for authentication, database connection, and rendering views.

I used the following command to install the dependencies:

npm install express ejs mysql2 dotenv bcrypt express-session

Explanation of each package:

• express – Main web framework for handling routes and requests.

• ejs – Template engine used to display dynamic content on web pages.

• mysql2 – Used to connect Node.js with MySQL database on Railway.

• dotenv – Helps manage secret environment variables securely (.env file).

• bcrypt – Used to hash passwords securely before saving them to the database.

• express-session – Used to manage user login sessions.

These dependencies create the foundation for authentication (register + login), database connection, and secure routing. They are required for building a full Node.js web application with login system.

Additional useful dependencies (optional):

npm install nodemon --save-dev

nodemon helps restart the server automatically while coding. This is useful during development but not required in production.

After installation, all installed packages were automatically added inside the *package.json* file under the **dependencies** section.

```
up to date, audited 101 packages in 2s

17 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

8. Deployment of Node.js Application to Linux Server Using SSH

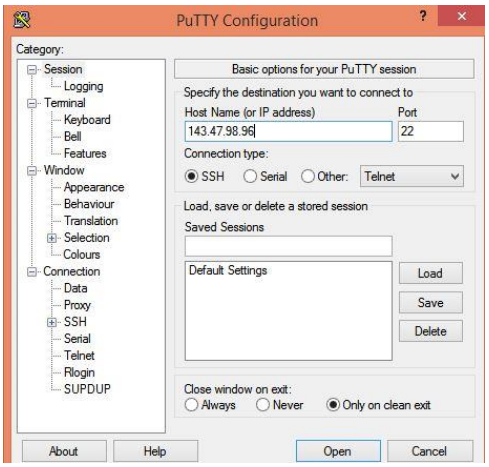
This section explains how the Node.js authentication web application was deployed from a GitHub repository to a Linux server using **Secure Shell (SSH)**. The deployment was done without root (sudo) access, which is common in school and shared servers.

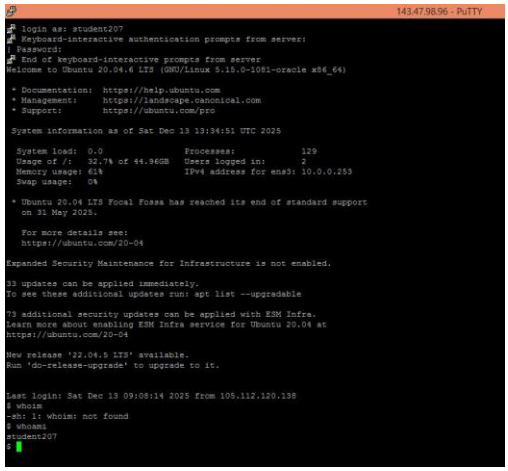
8.1 Accessing the Linux Server via SSH

The Linux server was accessed remotely using SSH from the terminal. SSH provides a secure way to connect to a remote server.

```
ssh student207@143.47.98.99
```

After successful authentication, access to the Linux terminal was granted.





8.2 Checking Existing Node.js Installation

After logging into the server, the installed Node.js version was checked:

```
node -v
npm -v
```

The server was running an outdated Node.js version, which is not suitable for modern Node.js applications.

```
student207
$ node -v
v22.19.0
$ npm -v
10.9.3
$
```

8.3 Installing Node.js Using NVM (Without Sudo)

Because the Linux server does not allow the use of `sudo`, **Node Version Manager (nvm)** was used to install Node.js locally in the user's home directory.

Installing NVM:

```
curl -o- https://raw.githubusercontent.com/nvm-
sh/nvm/v0.39.6/install.sh | bash
```

Loading NVM:

```
export NVM_DIR="$HOME/.nvm"
[ -s "$NVM_DIR/nvm.sh" ] && . "$NVM_DIR/nvm.sh"
```

Installing Node.js:

```
nvm install 20
nvm use 20
```

Verifying Installation:

```
node -v
npm -v
```

8.4 Cloning the Project from GitHub

The Node.js project was uploaded to GitHub and cloned directly into the Linux server using Git.

```
git clone https://github.com/intelstests/nodeview.git
```

After cloning, the project directory was accessed:

```
cd repository-name
ls
```

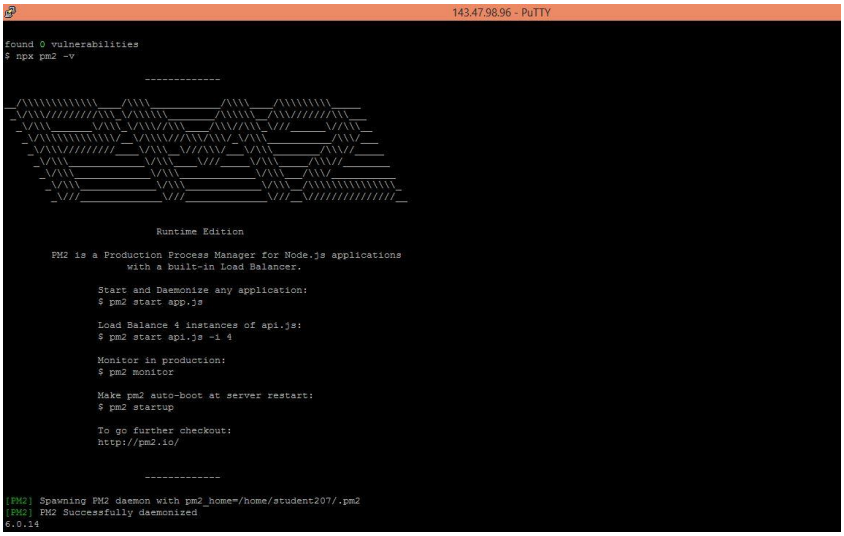
The `package.json` file confirmed that the project was cloned successfully.

8.5 Installing Project Dependencies on the Server

Inside the project directory, all required dependencies were installed using npm:

```
npm install
```

This command installed all packages listed in the `package.json` file and created the `node_modules` directory.



8.6 Environment Variable Configuration on Linux Server

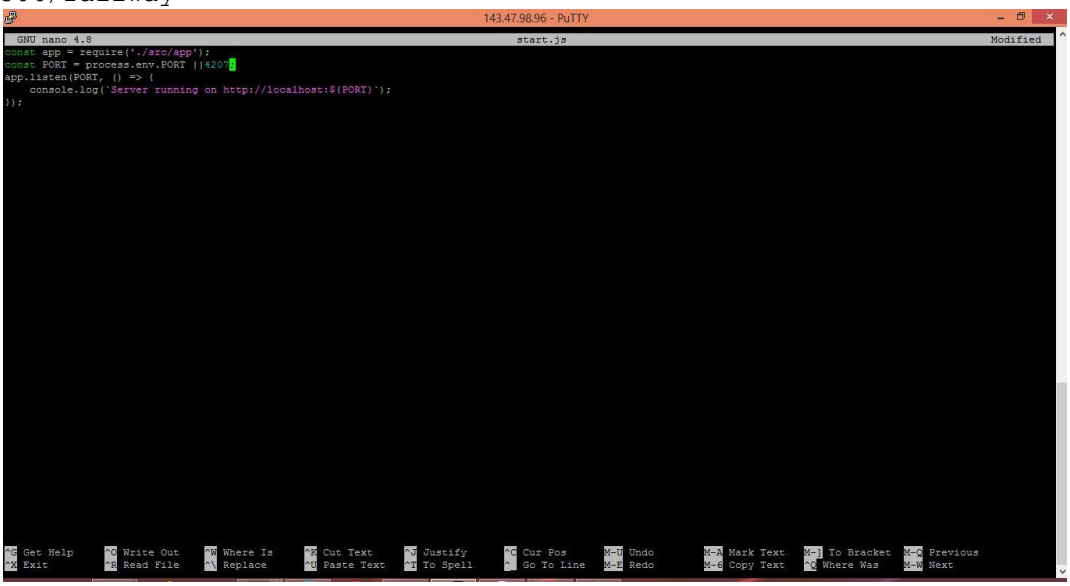
An environment file was created on the Linux server to store sensitive configuration values such as database credentials.

```
nano .env
```

Example configuration:

```
PORT=4207
DATABASE_URL
=mysql://root:iEhWTGJTGPmPzVrlaylpBOftTxyZSupZ@mysql.railway.internal:3
```

```
306/railway
```

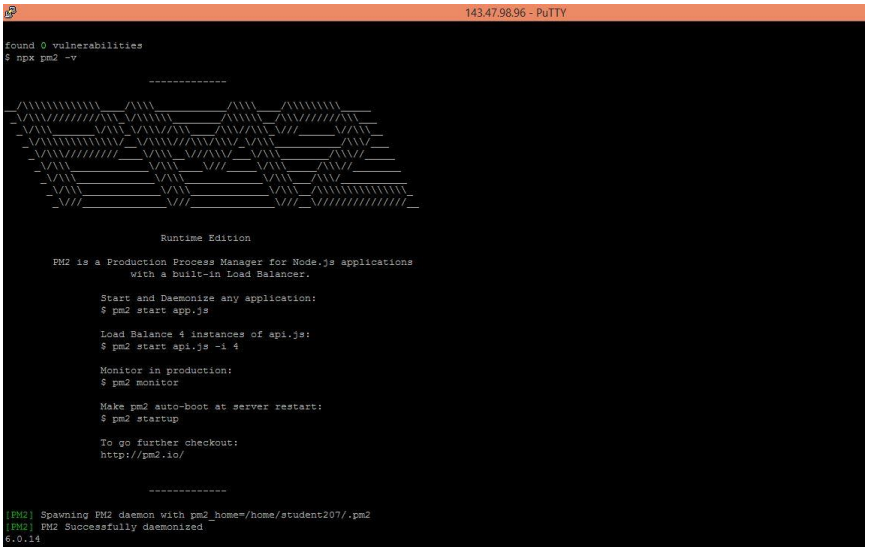


8.7 Installing PM2 Process Manager

To keep the Node.js application running in the background, **PM2** was installed globally using npm.

```
npm install pm2 --global
```

PM2 helps manage and monitor Node.js applications on a production server.



8.8 Starting the Application Using PM2

The application was started using PM2 with a custom process name:

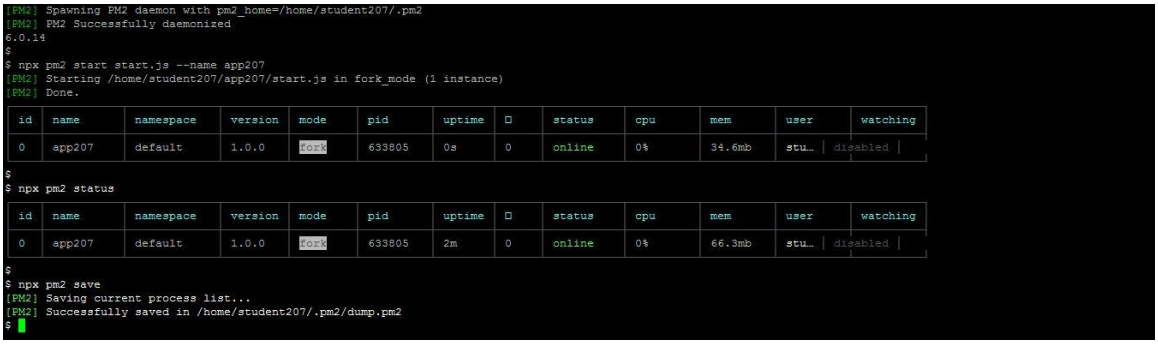
```
pm2 start start.js --name app207
```

The running status of the application was verified using:

```
pm2 list
```

Application logs were viewed using:

```
pm2 logs app207
```

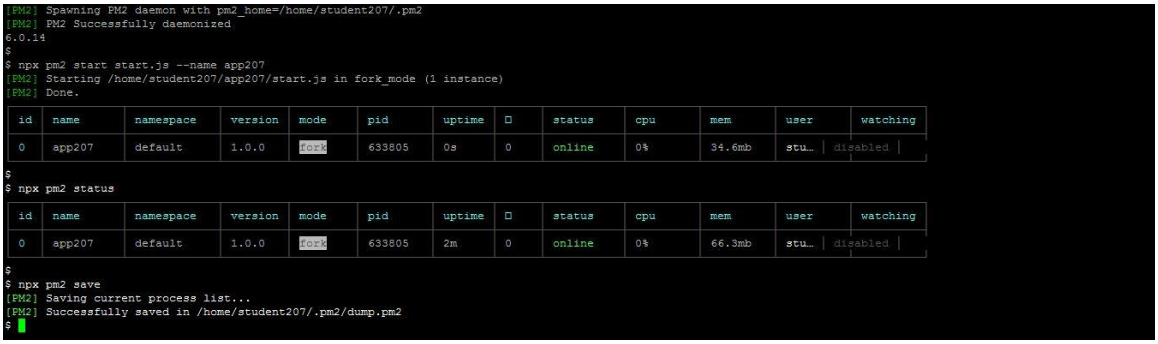


8.9 Saving PM2 Process for Auto Restart

To ensure the application restarts automatically after server reboot, the PM2 process list was saved:

```
pm2 save
pm2 startup
```

This step ensures application availability and reliability.



8.10 Deployment Summary

The Node.js authentication web application was successfully deployed on a Linux server using SSH and GitHub. By using NVM and PM2, the application runs without requiring administrative privileges and remains active even after server restarts.

This deployment demonstrates practical knowledge of:

- Linux server usage
- Secure remote access using SSH
- GitHub-based project deployment
- Node.js environment management
- Production process management using PM2